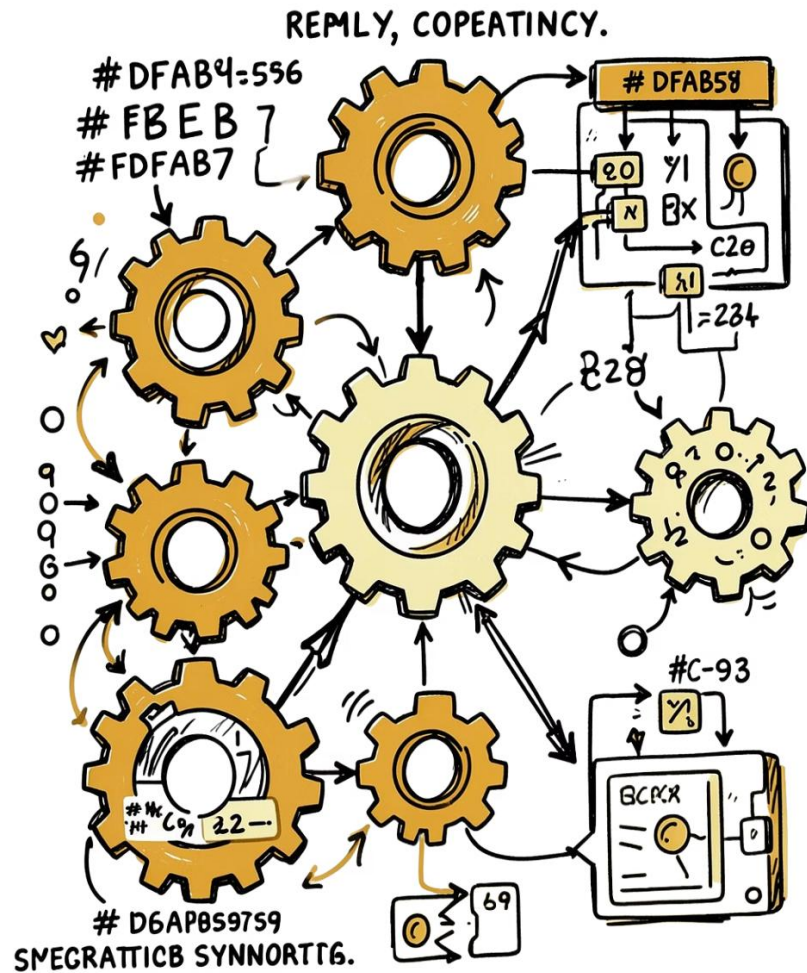
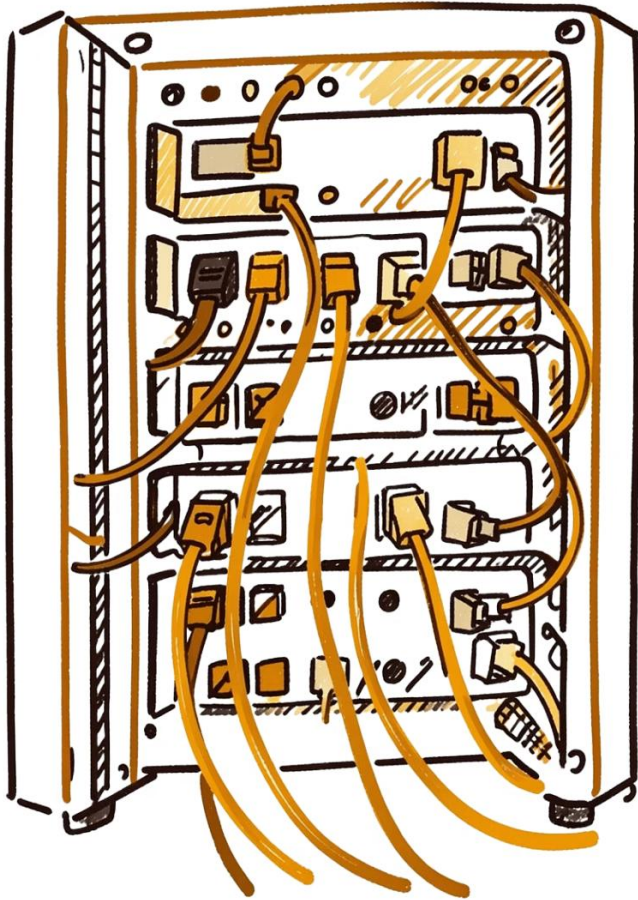




Priority Ceiling Protocol - Resource Access Protocols

A study of concurrency control mechanisms in real-time systems — from basic semaphores to advanced protocols that bound priority inversion and guarantee schedulability.

PREPARED BY CHIMEZIE DANIEL CHIDI 1230515



Resources, Critical Sections & Semaphores

Private Resource

Dedicated to a single process.

Shared Resource

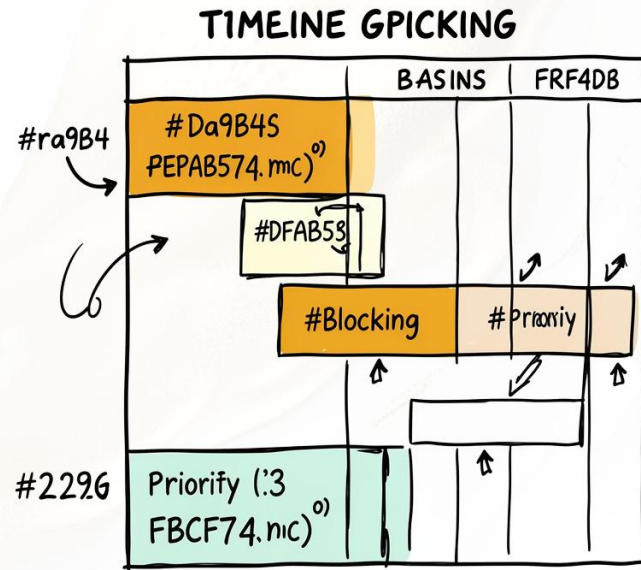
Accessible by multiple tasks.

Exclusive Resource

Shared but protected against concurrent access.

A **critical section** is code executed under mutual exclusion. Tasks use **semaphores** via *wait/signal* primitives. Blocked tasks enter a **WAIT** state, then return to **READY** (not RUN) when signaled.

The Priority Inversion Phenomenon



What Is It?

A high-priority task is forced to wait for lower-priority tasks holding a shared resource — potentially **unboundedly**.

Classic Scenario

τ_3 holds a resource $\rightarrow \tau_1$ preempts and blocks $\rightarrow \tau_2$ (medium priority) preempts τ_3 , extending τ_1 's wait beyond τ_3 's critical section.



Any intermediate-priority task can prolong blocking, causing critical deadlines to be missed.

Five Resource Access Protocols

01

Non-Preemptive Protocol (NPP)

Disables preemption inside all critical sections.

02

Highest Locker Priority (HLP)

Raises priority to the resource's *ceiling* on entry.

03

Priority Inheritance Protocol (PIP)

Blocking tasks inherit the highest blocked priority.

04

Priority Ceiling Protocol (PCP)

Denies lock if priority $\leq$ any locked semaphore's ceiling.

05

Stack Resource Policy (SRP)

Blocks at preemption time; supports EDF and stack sharing.

Non-Preemptive & Highest Locker Priority

NPP

Raises task priority to the system maximum inside any critical section. Simple, but causes **unnecessary blocking** — even high-priority tasks not sharing the resource are blocked.

Max blocking: longest critical section of any lower-priority task.

HLP / IPC

Raises priority to the **resource ceiling** — the highest priority of tasks sharing that resource.

Bounds blocking to **one critical section**. More precise than NPP, but may block before the resource is actually needed.

Priority Inheritance Protocol (PIP)

Direct Blocking

High-priority task waits for a lower-priority task holding a needed resource.

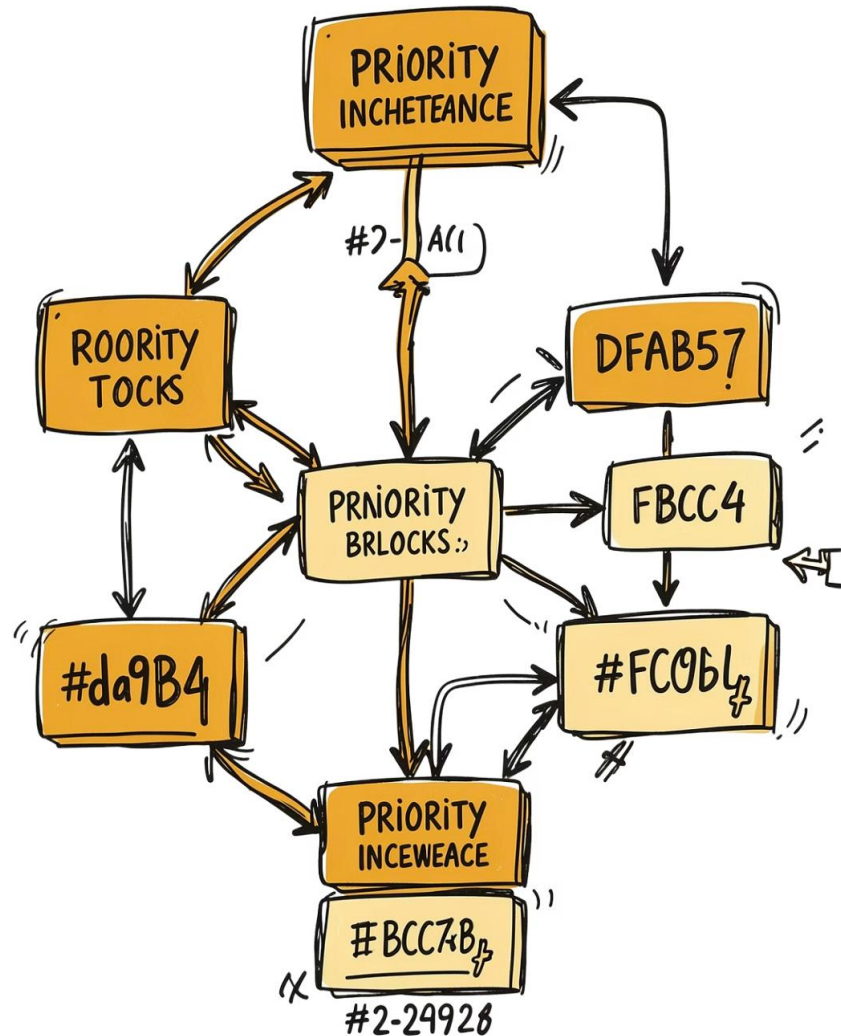
Push-Through Blocking

Medium-priority task blocked by a low-priority task that inherited a higher priority.

Transitive Inheritance

If τ_3 blocks τ_2 and τ_2 blocks τ_1 , then τ_3 inherits τ_1 's priority via τ_2 .

Under PIP, τ_i can be blocked at most $\alpha_i = \min(l_i, s_i)$ critical sections, where l_i = lower-priority tasks that can block, s_i = distinct blocking semaphores.



PIP: Chained Blocking & Deadlock

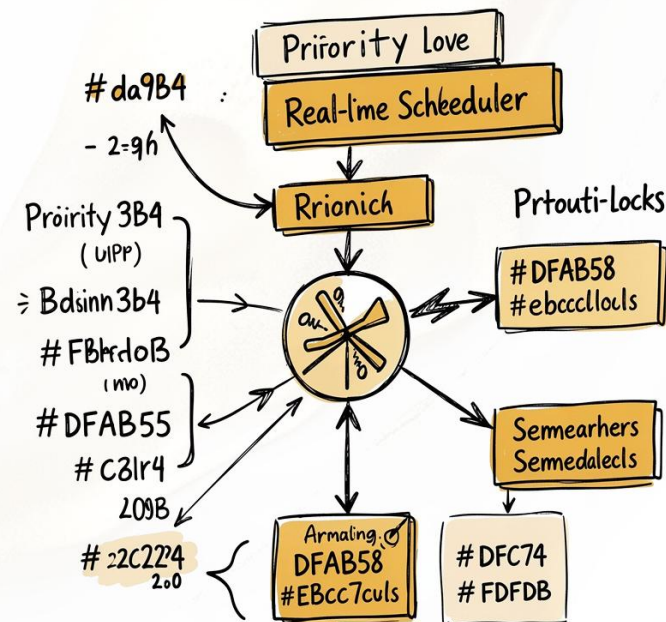
Chained Blocking

If τ_1 accesses n semaphores each locked by a different lower-priority task, it can be blocked for **n critical sections** — still bounded, but potentially substantial.

Deadlock

PIP does **not** prevent deadlocks. Two tasks nesting semaphores in reverse order can deadlock. Solution: impose a **total ordering** on semaphore accesses.

Priority Ceiling Protocol (PCP)



Core Rule

A task may enter a critical section only if its priority is **higher** than the ceiling of all semaphores currently locked by other tasks.

Key Properties

- Blocks at most **one** critical section
- **Prevents deadlocks** by construction
- **Prevents chained blocking**
- Introduces **ceiling blocking** as a third form

Stack Resource Policy (SRP)

Preemption Test

Task τ may run only if $\pi(\tau) > \Pi_s$ (system ceiling). Blocks at **preemption time**, not access time.

Dynamic Priorities

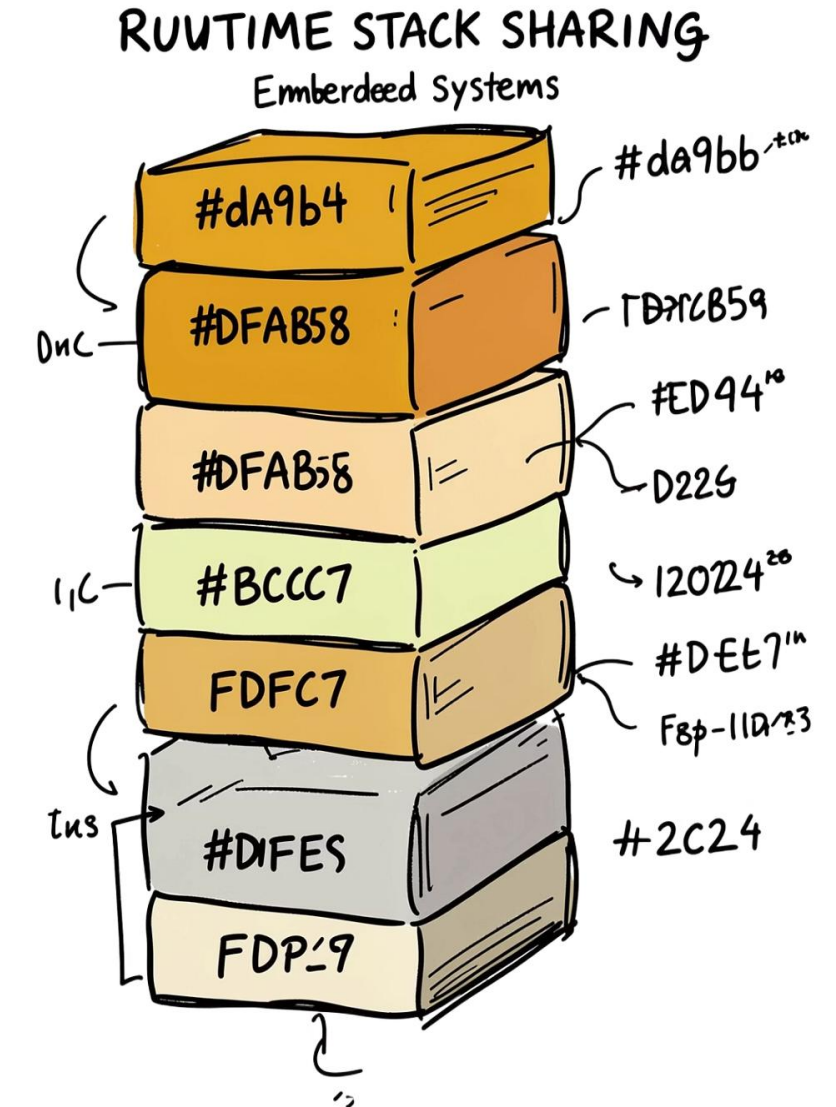
Supports both fixed-priority (RM) and dynamic (EDF) scheduling natively.

Stack Sharing

Tasks share a single stack — saves up to **90%** memory with many tasks across few preemption levels.

Multi-Unit Resources

Generalizes PCP to resources with multiple concurrent units.



PCP_UPPAAL Analysis

PCP_Submission_Ready.xml - UPPAAL

File Edit View Tools Options Help

Editor Symbolic Simulator Concrete Simulator Verifier

Overview

A[] not deadlock
A[] not (T1.Critical and T2.Critical and owner[R1] = owner[R2])
A[] not (T1.Critical and T3.Critical)
A[] T1.Critical imply owner[R1] = 1
A[] T2.Critical imply owner[R2] = 2
A[] T3.Critical imply owner[R1] = 3
E<> T1.Critical
E<> T2.Critical
E<> T3.Critical
A[] locked[R1] imply owner[R1] ≠ FREE
A[] locked[R2] imply owner[R2] ≠ FREE

Protocol	Definition	Blocking	Deadlock Free	Priority Type	Complexity
NPP (Non-Preemptive Protocol)	Disables preemption while inside a critical section.	1 CS	✓	Any	Easy
HLP (Highest Locker Priority)	Task inherits the ceiling priority of the resource it locks.	1 CS	✓	Fixed	Easy
PIP (Priority Inheritance Protocol)	Resource holder inherits the priority of the highest blocked task.	α_i	✗	Fixed	Hard
PCP (Priority Ceiling Protocol)	Task can lock a resource only if its priority exceeds the current system ceiling.	1 CS	✓	Fixed	Medium
SRP (Stack Resource Policy)	Blocks tasks at preemption time using a system ceiling; supports stack sharing.	1 CS	✓	Any (RM/EDF)	Easy

APPLICATIONS

Industry

 Aerospace

 Automotive

 Medical

 Industrial Automation

 Rail Systems

 Drones

 Power Systems

 Space Systems

Example

Flight Control Computers

ABS, Airbag, Engine Control Units

Ventilators, Infusion Pumps, Patient Monitors

Robotic Arms, CNC Machines, Production Lines

Train Signaling and Control

Flight Stabilization and Navigation

Substation Protection Relays

Satellite Control Software

THANK YOU !

EXTRA – FOR MY PERSONAL REFERENCE